



BACKEND

RUMO AO FULL STACK



por Iuri Silva

Atenção!

 **O material de backend ainda está em construção!**

Em breve, você terá acesso a novos conteúdos sobre Java.

Fique tranquilo(a): para receber as atualizações, basta acessar novamente este material pela plataforma da Hotmart, sem custo adicional. 

Quero agradecer pela confiança nos meus materiais . Aos poucos, estou aprimorando cada detalhe para entregar o melhor para você.

Se tiver alguma sugestão, será um prazer ouvir sua opinião! É só me chamar no Instagram [@iuricode](https://www.instagram.com/iuricode)



A próxima atualização já tem data marcada: dia 01/03/2026, com conteúdo novo de Java.



MÓDULO 1

Python

Cada linha de código é invisível para o usuário, mas essencial para o mundo digital funcionar. O que você constrói hoje nos bastidores pode sustentar o futuro da tecnologia.

O que é

Python é uma linguagem de programação de alto nível, interpretada e extremamente versátil. É usada em diversas áreas como desenvolvimento web, ciência de dados, inteligência artificial, automação, entre outras.

Python tem uma sintaxe simples tornando-o mais fácil de aprender e ler. Além disso, Python tem uma vasta quantidade de bibliotecas e frameworks, como o numpy para cálculo numérico, pandas para manipulação de dados, e django para desenvolvimento web.

Instalação

Antes de começar a programar, precisamos instalar o Python no seu computador. Siga os passos abaixo para configurar seu ambiente:

1. Acesse o [site oficial do Python](#).
2. Baixe a versão mais recente para o seu sistema operacional (Windows, macOS ou Linux).
3. Durante a instalação, **não se esqueça de marcar a opção "Add Python to PATH"**! Isso permitirá que você execute o Python diretamente no terminal.

Abra o terminal (no Windows, use o Prompt de Comando ou Powershell; no macOS e Linux, o Terminal) e digite o seguinte comando:



```
python --version
```

Se tudo estiver certo, a versão do Python será exibida.

Hello, World!

Vamos criar nosso primeiro programa em Python: o famoso Hello, World!. Abra seu editor de texto (pode ser o VSCode, PyCharm ou até o bloco de notas, se preferir) e escreva o seguinte código:



```
print("Hello, World!")
```

Agora, salve o arquivo com o nome `olamundo.py` e execute no terminal com o comando:



```
python olamundo.py
```

Se tudo estiver certo, você verá o seguinte no terminal:



```
Hello, World!
```

Como executar

Existem várias maneiras de executar um script Python, e conhecer todas elas é importante para aumentar a sua eficiência como programador.

Executando pelo terminal

Como você acabou de ver, podemos executar um script diretamente pelo terminal. Basta navegar até o diretório onde o arquivo está salvo e usar o comando:



```
python nome_do_arquivo.py
```

Fundamentos da linguagem

Tipos de dados básicos (int, float, str, bool)

O Python possui tipos de dados essenciais que formam a base da linguagem. Vamos entendê-los e como eles se comportam.

Inteiros

Os números inteiros são aqueles sem casas decimais, como 1, 42, -10. Em Python, eles são representados pelo tipo int.



```
meu_inteiro = 10  
print(type(meu_inteiro)) # <class 'int'>
```

Pontos flutuantes

Os floats são números com casas decimais. Por exemplo, 3.14 e -7.0. Eles são cálculos mais precisos, como medições e valores monetários.



```
meu_float = 3.14  
print(type(meu_float)) # <class 'float'>
```

Imagine que você está medindo a quantidade de água em um copo. Você pode ter 150 ml ou 150.5 ml. O primeiro é um int, e o segundo é um float.

Strings

Uma string é uma sequência de caracteres, usada para representar texto. Pode ser qualquer coisa que você coloca entre aspas simples ' ' ou duplas " ".

```
meu_texto = "Olá, Python!"  
print(type(meu_texto)) # <class 'str'>
```

Booleanos

O tipo bool é usado para representar valores verdadeiros ou falsos, com apenas dois possíveis valores: True ou False. Eles são usados em estruturas de controle de fluxo, como condicionais.

```
meu_bool = True  
print(type(meu_bool)) # <class 'bool'>
```

Variáveis e tipos de atribuição

As variáveis são como "caixas" onde você armazena valores. Elas permitem que você manipule dados de forma dinâmica durante a execução do seu programa.

Como declarar variáveis

Você não precisa especificar o tipo da variável, o Python é dinamicamente tipado. Basta atribuir um valor com =.



```
idade = 25    # 'idade' agora contém um inteiro  
nome = "Iuri" # 'nome' agora contém uma string
```

Regras para nomear variáveis

- Devem começar com uma letra ou _ (underline).
- Não podem começar com um número.
- Não podem conter espaços ou caracteres especiais como @, #.
- São case-sensitive (idade e Idade são variáveis diferentes).

Entrada e saída de dados (input e print)

A comunicação entre você e seu programa é feita com as funções input() e print().

Saída de dados com print()

A função print() é usada para exibir informações na tela.



```
print("Olá, Mundo!")
```

Entrada de dados com input()

A função input() permite que o usuário insira dados. O valor inserido é sempre do tipo string.



```
idade = int(input("Digite sua idade: "))  
print("Você tem", idade, "anos.")
```

Operadores aritméticos

Esses operadores permitem realizar operações matemáticas:



Operador	Descrição	Exemplo
<code>`+`</code>	Adição	<code>`2 + 3` = 5</code>
<code>`-`</code>	Subtração	<code>`5 - 2` = 3</code>
<code>`*`</code>	Multiplificação	<code>`3 * 4` = 12</code>
<code>`/`</code>	Divisão	<code>`10 / 2` = 5</code>
<code>`//`</code>	Divisão inteira	<code>`10 // 3` = 3</code>
<code>`%`</code>	Módulo	<code>`10 % 3` = 1</code>
<code>`**`</code>	Potência	<code>`2 ** 3` = 8</code>

Operadores aritméticos

Esses operadores permitem realizar operações matemáticas:



Operador	Descrição	Exemplo
<code>`==`</code>	Igual	<code>`5 == 5` = True</code>
<code>`!=`</code>	Diferente	<code>`5 != 3` = True</code>
<code>`>`</code>	Maior que	<code>`5 > 3` = True</code>
<code>`<`</code>	Menor que	<code>`3 < 5` = True</code>
<code>`>=`</code>	Maior ou igual	<code>`5 >= 5` = True</code>
<code>`<=`</code>	Menor ou igual	<code>`3 <= 5` = True</code>

Operadores lógicos

São usados para combinar expressões:



Operador	Descrição	Exemplo
<code>`and`</code>	E	<code>`True and False` = False</code>
<code>`or`</code>	Ou	<code>`True or False` = True</code>
<code>`not`</code>	Não	<code>`not True` = False</code>

Controle de fluxo

Condicionais (if, elif, else)

As estruturas condicionais permitem que o programa tome decisões com base em certas condições, assim como nós fazemos diariamente.

Imagine que você está planejando sair, e decide se vai levar um guarda-chuva. Se estiver chovendo, você leva; se não, deixa em casa. As condicionais no Python funcionam da mesma forma.

Estrutura if

A estrutura if é usada para executar um bloco de código se uma condição for verdadeira.



```
idade = 18
if idade ≥ 18:
    print("Você é maior de idade.")
```

Estrutura else

O else especifica o que fazer caso a condição do if não seja verdadeira.



```
idade = 16
if idade ≥ 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

Estrutura elif

Se você precisar verificar várias condições, pode usar elif (abreviação de "else if"). Pense nele como opções adicionais para o seu "se".



```
nota = 8.5
if nota ≥ 9:
    print("Excelente!")
elif nota ≥ 7:
    print("Bom trabalho!")
else:
    print("Você precisa melhorar.")
```

Estruturas de repetição (for e while)

As estruturas de repetição permitem executar um bloco de código várias vezes, o que é útil para evitar repetições desnecessárias.

Estrutura for

O for é usado quando sabemos o número de iterações ou quando queremos percorrer uma sequência (lista, tupla, string, etc.).



```
nomes = ["Iuri", "Vitoria", "Amora"]
for nome in nomes:
    print("Olá,", nome)
```

Função range() com for

O range() gera uma sequência de números. Ele é muito útil em conjunto com for.

```
for i in range(5): # Imprime os números de 0 a 4
    print(i)
```

Estrutura while

O while é usado quando não sabemos quantas vezes a repetição deve ocorrer, pois ela depende de uma condição ser verdadeira.

```
contador = 0
while contador < 5:
    print("Contagem:", contador)
    contador += 1
```

Controle de loops (break e continue)

Às vezes, precisamos de mais controle sobre nossos loops. O Python oferece as palavras-chave break, continue e pass para lidar com essas situações.

Break

O break encerra o loop imediatamente.



```
for numero in range(10):  
    if numero == 5:  
        break  
    print(numero)  
# Saída: 0, 1, 2, 3, 4
```

Continue

O continue interrompe a iteração atual e passa para a próxima.



```
for numero in range(5):  
    if numero == 2:  
        continue  
    print(numero)  
# Saída: 0, 1, 3, 4
```

Compreensão de listas (list comprehensions)

A compreensão de listas é uma maneira concisa de criar listas em Python, tornando seu código mais legível e eficiente.



```
numeros = [x for x in range(5)]  
print(numeros)  # [0, 1, 2, 3, 4]
```

Compreensão de listas com condições

Podemos adicionar condições à compreensão de listas.



```
pares = [x for x in range(10) if x % 2 == 0]  
print(pares)  # [0, 2, 4, 6, 8]
```

Estruturas de dados em Python

Listas

As listas são uma das estruturas de dados mais versáteis em Python. Pense nelas como uma prateleira de livros, onde cada livro ocupa uma posição específica e você pode acessar, adicionar ou remover livros quando quiser.

Criando listas

Você pode criar uma lista usando colchetes `[]` e separando os elementos por vírgulas.



```
minha_lista = [1, 2, 3, "Python", True]
print(minha_lista)  # [1, 2, 3, "Python", True]
```

Acessando elementos

Para acessar um elemento de uma lista, utilizamos o índice (posição) do item. Lembre-se que a contagem começa em 0.



```
print(minha_lista[0])  # 1
print(minha_lista[3])  # "Python"
```

Modificando listas

Você pode adicionar, remover ou alterar elementos da lista.

Adicionar: Use `append()` para adicionar um elemento ao final da lista.

```
minha_lista.append("Novo Item")  
print(minha_lista)
```

Remover: Use `remove()` para remover um item específico, ou `pop()` para remover pelo índice.

```
minha_lista.remove("Python")  
print(minha_lista)  
  
minha_lista.pop(0) # Remove o primeiro elemento  
print(minha_lista)
```

Iterando sobre listas

Você pode usar um `for` para percorrer cada item da lista.

```
for item in minha_lista:  
    print(item)
```

Tuplas

As tuplas são similares às listas, mas são imutáveis, ou seja, não podem ser alteradas depois de criadas. Pense nelas como o menu de um restaurante. Uma vez que o menu está impresso, não é possível alterar as opções sem fazer um novo.

Criando tuplas

Use parênteses () para criar uma tupla.

```
minha_tupla = (1, 2, 3, "Python", True)
print(minha_tupla)
```

Acessando elementos da tupla

O acesso é feito da mesma forma que em listas, utilizando índices.

```
print(minha_tupla[1]) # 2
```


Conjuntos (sets)

Conjuntos são coleções não ordenadas e sem elementos duplicados. Pense neles como uma caixa de lápis de cor, onde cada cor aparece apenas uma vez.

Criando conjuntos

```
Use chaves {} para criar um conjunto.  
meu_conjunto = {1, 2, 2, 3, 4, 4}  
print(meu_conjunto)  # {1, 2, 3, 4}
```

Operações com conjuntos

Você pode realizar operações como união (`|`), interseção (`&`), diferença (`-`).

```
a = {1, 2, 3}  
b = {3, 4, 5}  
  
print(a | b)  # União: {1, 2, 3, 4, 5}  
print(a & b)  # Interseção: {3}  
print(a - b)  # Diferença: {1, 2}
```

Dicionários

Os dicionários são como um manual de instruções, onde cada "palavra" tem um "significado". Eles armazenam pares de chave: valor.

Criando dicionários

Use chaves {} e separe os pares com :.

```
meu_dict = {  
    "nome": "Python", "ano": 1991,  
    "tipo": "Programação"  
}  
print(meu_dict)
```

Acessando e modificando valores

Você pode acessar um valor utilizando sua chave.

```
print(meu_dict["nome"]) # "Python"
```

Modificando

```
meu_dict["ano"] = 2025
print(meu_dict)
```

Métodos úteis de dicionários

- ``keys()``: Retorna todas as chaves.
- ``values()``: Retorna todos os valores.
- ``items()``: Retorna todos os pares chave-valor.

```
print(meu_dict.keys())
# dict_keys(['nome', 'ano', 'tipo'])
print(meu_dict.values())
# dict_values(['Python', 2025, 'Programação'])
```

Operações e métodos

Listas

- ``len(lista)``: Retorna o número de elementos.
- ``max(lista)``, ``min(lista)``: Retorna o maior e menor elemento.
- ``sort()``: Ordena a lista em ordem crescente.



```
lista = [3, 1, 4, 1, 5]
lista.sort()
print(lista)  # [1, 1, 3, 4, 5]
```

Tuplas

- `count(valor)`: Conta quantas vezes o valor aparece na tupla.
- `index(valor)`: Retorna o índice da primeira ocorrência do valor.

`index(valor)`: Retorna o índice da primeira ocorrência do valor.

Conjuntos

`add(elemento)`: Adiciona um elemento ao conjunto.

`remove(elemento)`: Remove um elemento específico.



```
meu_conjunto.add(6)
print(meu_conjunto)  # {1, 2, 3, 4, 6}
```

Dicionários

`get(chave)`: Retorna o valor associado à chave, ou None se não existir.

`pop(chave)`: Remove a chave e retorna seu valor.



```
valor = meu_dict.pop("ano")
print(valor) # 2024
print(meu_dict)
# {'nome': 'Python', 'tipo': 'Programação'}
```

Funções em Python

Definindo e chamando funções

As funções são blocos de código reutilizáveis que executam uma tarefa específica. Pense nelas como receitas de cozinha: você segue um conjunto de instruções para obter um resultado. Em Python, as funções permitem organizar e estruturar seu código, evitando repetições.

Criando uma função

Você define uma função usando a palavra-chave `def`, seguida do nome da função e parênteses `()`.



```
def saudacao():
    print("Olá, seja bem-vindo ao mundo do Python!")
```

Para chamar a função, basta usar seu nome seguido de parênteses:

```
saudacao()  
# Saída: Olá, seja bem-vindo ao mundo do Python!
```

Parâmetros e argumentos

As funções podem receber parâmetros, que são valores enviados para a função quando ela é chamada, permitindo que ela trabalhe de forma mais dinâmica.

Funções com parâmetros

```
def saudacao(nome):  
    print(f"Olá, {nome}, seja bem-vindo!")
```

Ao chamar a função, você passa o argumento:

```
saudacao("Iuri")  
# Saída: Olá, Iuri, seja bem-vindo!
```

Parâmetros com valor padrão

Você pode definir um valor padrão para um parâmetro. Se nenhum valor for fornecido, o padrão será usado.



```
def saudacao(nome="Visitante"):
    print(f"Olá, {nome}, seja bem-vindo!")
```

Retorno de valores

Uma função pode retornar um valor usando a palavra-chave return.



```
def soma(a, b):
    return a + b

resultado = soma(5, 3)
print(resultado) # Saída: 8
```

Funções embutidas (map, filter, zip, etc.)

Python oferece várias funções embutidas que facilitam a manipulação de coleções de dados, como map(), filter(), e zip().

Map()

A função `map()` aplica uma função a cada item de um iterável (como uma lista) e retorna um novo iterável com os resultados.

```
numeros = [1, 2, 3, 4]
dobrados = list(map(lambda x: x * 2, numeros))
print(dobrados) # Saída: [2, 4, 6, 8]
```

Filter()

A função `filter()` cria um iterável contendo apenas os itens que atendem a uma condição.

```
numeros = [1, 2, 3, 4, 5]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares) # Saída: [2, 4]
```

Zip()

A função `zip()` combina múltiplos iteráveis (listas, tuplas, etc.) em pares. É como fechar o zíper de uma jaqueta, onde os dentes de cada lado se unem.



```
nomes = ["Iuri", "Vitoria", "Amora"]
idades = [25, 30, 35]
combinacao = list(zip(nomes, idades))
print(combinacao)
# Saída: [('Iuri', 25), ('Vitoria', 21), ('Amora', 4)]
```

Módulos e pacotes

Módulos são arquivos que contêm definições e implementações de funções, classes e variáveis que você pode reutilizar em diferentes partes do seu código. Python possui muitos módulos padrão que você pode importar para ampliar suas funcionalidades, sem ter que reinventar a roda.

Importando módulos

Para importar um módulo, usamos a palavra-chave `import`. Por exemplo, o módulo `math` contém funções matemáticas avançadas:



```
import math
print(math.sqrt(16)) # Saída: 4.0
```

Importando apenas funções específicas

Você também pode importar apenas uma parte do módulo usando `from ... import ...`

```
from math import pi
print(pi) # Saída: 3.141592653589793
```

Renomeando módulos

Você pode usar a palavra-chave `as` para renomear um módulo ou função para simplificar seu uso.

```
import math as m
print(m.sqrt(25)) # Saída: 5.0
```

Criando seus próprios módulos

Assim como o Python oferece módulos, você também pode criar os seus. Basta criar um arquivo `.py` com funções e variáveis que deseja reutilizar.

Exemplo de módulo personalizado

Crie um arquivo chamado `meu_modulo.py` com o seguinte conteúdo:

```
def saudacao(nome):  
    return f"Olá, {nome}!"
```

Depois, em outro arquivo Python, você pode importar e usar o módulo:

```
import meu_modulo  
print(meu_modulo.saudacao("Iuri")) # Saída: Olá, Iuri!
```

Uso de pacotes externos

Python possui um gerenciador de pacotes chamado `pip` que permite instalar e gerenciar pacotes externos. Pacotes são coleções de módulos organizados para resolver problemas específicos.

Instalando um pacote com pip

Para instalar um pacote, basta usar o comando `pip install nome_do_pacote`. Por exemplo, para instalar o pacote `requests`, que permite fazer requisições HTTP, use:



```
pip install requests
```

Após a instalação, você pode importar o pacote normalmente:



```
import requests  
response = requests.get("https://api.github.com")  
print(response.status_code) # Saída: 200
```

Existem inúmeras bibliotecas em Python que facilitam o desenvolvimento em diferentes áreas. Aqui estão algumas das mais populares para iniciantes:

- **NumPy**: Trabalhando com arrays e operações matemáticas.
- **Pandas**: Manipulação e análise de dados.

Manipulação de arquivos

Python oferece uma série de ferramentas para ler e escrever em arquivos, o que é extremamente útil para armazenar e processar dados. Pense nos arquivos como cadernos onde você pode registrar e consultar informações a qualquer momento.

Leitura e escrita de arquivos de texto

A manipulação de arquivos de texto em Python é feita utilizando as funções `open()`, `read()`, `write`, e `close()`.

Abrindo um arquivo

Para abrir um arquivo, usamos a função `open()`, que requer o nome do arquivo e o modo de abertura:

- `'r'`: leitura (`read`)
- `'w'`: escrita (`write`) – cria um novo arquivo ou substitui o existente
- `'a'`: adicionar (`append`) – adiciona conteúdo ao final do arquivo existente



```
arquivo = open("exemplo.txt", "w")
arquivo.write("Olá, Python!")
arquivo.close()
```

Lendo um arquivo

Para ler o conteúdo de um arquivo, use o método `read()`.



```
arquivo = open("exemplo.txt", "r")
conteudo = arquivo.read()
print(conteudo) # Saída: Python!
arquivo.close()
```

Manipulação de arquivos CSV

Arquivos CSV (Comma-Separated Values) são comumente usados para armazenar dados tabulares. Python possui o módulo `csv` para facilitar a leitura e escrita desses arquivos.

Lendo um arquivo CSV

```
import csv

with open("dados.csv", "r") as arquivo_csv:
    leitor = csv.reader(arquivo_csv)
    for linha in leitor:
        print(linha)
```

Escrevendo em um arquivo CSV

```
import csv

dados = [
    ["Nome", "Idade"], ["Iuri", 25], ["Vitoria", 21]
]

with open("dados.csv", "w", newline="") as arquivo_csv:
    escritor = csv.writer(arquivo_csv)
    escritor.writerows(dados)
```

Uso da biblioteca JSON

O JSON (JavaScript Object Notation) é um formato de armazenamento de dados muito utilizado para comunicação entre sistemas. Python possui o módulo json para manipular esses arquivos.

Lendo um arquivo JSON



```
import json

with open("dados.json", "r") as arquivo_json:
    dados = json.load(arquivo_json)
    print(dados)
```

Escrevendo em um arquivo JSON



```
import json

dados = {"nome": "Turi", "cidade": "Garça"}

with open("dados.json", "w") as arquivo_json:
    json.dump(dados, arquivo_json)
```


Trabalhando com arquivos binários

Além de arquivos de texto e CSV, você pode manipular arquivos binários, como imagens, arquivos executáveis, entre outros.

Lendo um arquivo binário



```
with open("imagem.png", "rb") as arquivo_binario:
    conteudo = arquivo_binario.read()
    print(conteudo)
```

Escrevendo em um arquivo binário



```
with open("copia_imagem.png", "wb") as arquivo_binario:
    arquivo_binario.write(conteudo)
```

Tratamento de exceções

O tratamento de exceções é uma parte crucial da programação, pois permite que seu programa lide de forma inteligente com erros que possam ocorrer durante a execução. Em vez de deixar o programa "quebrar", você pode capturar esses erros e tomar ações apropriadas.

Erros em Python podem ser classificados em dois tipos:

- Erros de sintaxe: Erros na forma como o código é escrito, impedindo a execução.
- Exceções: Erros que ocorrem durante a execução do código, mesmo que a sintaxe esteja correta (por exemplo, dividir por zero).

Exemplo de uma exceção:



```
numero = int(input("Digite um número: "))  
print(10 / numero)  
# Se digitar 0, ocorrerá um erro "ZeroDivisionError"
```

Blocos try, except, else e finally

Usando try e except

O bloco try permite que você execute um código que pode gerar uma exceção. O bloco except captura e lida com a exceção, evitando que o programa seja interrompido.



```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
    print(resultado)
except ZeroDivisionError:
    print("Erro: Divisão por zero não é permitida.")
except ValueError:
    print("Erro: Você deve digitar um número válido.")
```

Usando else

O bloco else é executado apenas se nenhuma exceção for gerada no bloco try.



```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
except ZeroDivisionError:
    print("Erro: Divisão por zero não é permitida.")
else:
    print("Resultado:", resultado)
```

Usando finally

O bloco finally é executado independentemente de ocorrer uma exceção ou não. Ele é útil para liberar recursos, como fechar arquivos.



```
try:
    arquivo = open("dados.txt", "r")
    conteudo = arquivo.read()
except FileNotFoundError:
    print("Erro: Arquivo não encontrado.")
finally:
    arquivo.close()
    print("Arquivo fechado.")
```



MÓDULO 2

Bancos de dados

Cada linha de código é invisível para o usuário, mas essencial para o mundo digital funcionar. O que você constrói hoje nos bastidores pode sustentar o futuro da tecnologia.

O que é

Um banco de dados é uma coleção organizada de dados que pode ser facilmente acessada, gerenciada e atualizada. Em termos simples, é um lugar onde você armazena suas informações de forma estruturada para que possa recuperá-las e manipulá-las conforme necessário.

Mais usados

Relacionais:

- Exemplo: MySQL, PostgreSQL, SQLite.
- Características: Utilizam tabelas para armazenar dados, onde cada tabela é uma coleção de registros. Usam SQL (Structured Query Language) para manipulação e consulta de dados.

```
CREATE TABLE Usuarios (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100),  
  email VARCHAR(100)  
);
```

Não relacionais (NoSQL):

- Exemplo: MongoDB, Cassandra, Redis.
- Características: Não usam tabelas relacionais. Podem usar documentos, pares chave-valor, colunas largas ou grafos para armazenar dados. Mais flexíveis em termos de estrutura de dados.

```
{  
  "id": 1,  
  "nome": "Iuri",  
  "email": "iuri@email.com"  
}
```

Linguagem SQL

SQL (Structured Query Language) é a linguagem padrão para interagir com bancos de dados relacionais. Com SQL, você pode criar, ler, atualizar e excluir dados em um banco de dados. É essencial para qualquer pessoa que queira trabalhar com bancos de dados relacionais.

Tipos de comandos

1. DDL (Data Definition Language): Usado para definir e modificar a estrutura do banco de dados. Exemplos: CREATE, ALTER, DROP.



```
CREATE TABLE Produtos (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100),  
    preco DECIMAL(10, 2)  
);
```

2. DML (Data Manipulation Language): Usado para manipular os dados dentro das tabelas. Exemplos: INSERT, UPDATE, DELETE.



```
INSERT INTO Produtos (id, nome, preco)  
VALUES (1, 'Notebook', 2999.99);
```

3. DCL (Data Control Language): Usado para controlar o acesso aos dados no banco de dados. Exemplos: GRANT, REVOKE.



```
GRANT SELECT ON Produtos TO usuario;
```


Tipos de dados

Aprender os tipos de dados que uma tabela pode ter é crucial. Aqui estão os tipos de dados mais comuns em SQL:

- INT: Números inteiros. Exemplo: INT, SMALLINT, BIGINT.
- FLOAT, DOUBLE: Números de ponto flutuante. Exemplo: FLOAT, DOUBLE.
- DECIMAL: Números decimais precisos. Exemplo: DECIMAL(10, 2) (10 dígitos no total, 2 após o ponto decimal).
- CHAR, VARCHAR: Strings de comprimento fixo e variável. Exemplo: CHAR(10), VARCHAR(255).
- TEXT: Texto longo.
- DATE, TIME, DATETIME, TIMESTAMP: Data e hora. Exemplo: DATE, TIME, TIMESTAMP.
- BOOLEAN: Valores booleanos (TRUE ou FALSE).

Como rodar o SQL

Se você quer praticar SQL sem precisar configurar um servidor de banco de dados localmente, recomendo utilizar o [SQL Fiddle](#).

Depois de acessar basta selecionar o tipo de banco de dados que você quer usar (por exemplo, MySQL).

Aqui está um comando SQL para você testar:

```
CREATE TABLE Produto (  
    ID INT AUTO_INCREMENT KEY,  
    Nome VARCHAR(100),  
    Tipo VARCHAR(255)  
);  
  
INSERT INTO Produto(Nome, Tipo)  
VALUES ('Arroz', 'Integral');  
  
SELECT * FROM Produto;
```

Criação e manipulação de tabelas

Comandos CREATE, ALTER, DROP

Os comandos DDL (Data Definition Language) são usados para definir e modificar a estrutura das tabelas no banco de dados. Vamos explorar cada um deles em detalhes.

CREATE TABLE

O comando CREATE TABLE é usado para criar novas tabelas no banco de dados.



```
CREATE TABLE NomeDaTabela (  
    coluna1 tipo_dado restrições,  
);
```

Exemplo:



```
CREATE TABLE Usuarios (  
    nome VARCHAR(100) NOT NULL,  
);
```

ALTER TABLE

O comando ALTER TABLE é usado para modificar a estrutura de uma tabela existente. Você pode adicionar, modificar ou remover colunas, bem como adicionar ou remover restrições.

Adicionar coluna:



```
ALTER TABLE Usuarios ADD telefone VARCHAR(15);
```

Modificar coluna:



```
ALTER TABLE Usuarios  
MODIFY email VARCHAR(150) NOT NULL;
```

Remover coluna:



```
ALTER TABLE Usuarios DROP COLUMN estado;
```

DROP TABLE

O comando DROP TABLE é usado para excluir uma tabela e todos os seus dados do banco de dados.



```
DROP TABLE Usuarios;
```

Chaves primárias e estrangeiras

PRIMARY KEY

A chave primária é uma coluna ou conjunto de colunas que identifica exclusivamente cada registro na tabela.

```
CREATE TABLE Clientes (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE  
);
```

FOREIGN KEY

A chave estrangeira é uma coluna que cria uma relação entre duas tabelas. Ela deve referenciar uma chave primária em outra tabela.

```
CREATE TABLE Pedidos (  
  id INT PRIMARY KEY,  
  cliente_id INT,  
  FOREIGN KEY (cliente_id) REFERENCES Clientes(id)  
);
```

UNIQUE

Garante que todos os valores em uma coluna sejam distintos.



```
CREATE TABLE Usuarios (  
    id INT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) UNIQUE  
);
```

NOT NULL

Garante que uma coluna não pode conter valores nulo



```
CREATE TABLE Produtos (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10, 2) NOT NULL  
);
```

CHECK

Garante que todos os valores em uma coluna satisfaçam uma condição específica.

```
CREATE TABLE Funcionarios (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    salario DECIMAL(10, 2) CHECK (salario > 0)  
);
```

Inserção, atualização e exclusão de dados

Comandos INSERT, UPDATE, DELETE

A manipulação de dados é uma das operações mais comuns em SQL. Vamos explorar como inserir, atualizar e excluir dados nas tabelas de um banco de dados.

INSERT INTO

O comando INSERT INTO é usado para adicionar novos registros a uma tabela.

Sintaxe:



```
INSERT INTO NomeDaTabela (coluna1, coluna2, ... )  
VALUES (valor1, valor2, ... );
```

Exemplo:



```
INSERT INTO Clientes (id, nome, email, data_registro)  
VALUES (1, 'Iuri', 'iuri@email.com', '2025-07-02');
```

Você também pode inserir vários registros de uma vez:



```
INSERT INTO Clientes (id, nome, email, data_registro)  
VALUES  
(2, 'Vitória', 'vitoria@email.com', '2025-07-03'),  
(3, 'Amora', 'amora@email.com', '2025-07-03');
```

UPDATE

O comando UPDATE é usado para modificar os dados existentes em uma tabela.

Sintaxe:



```
UPDATE NomeDaTabela  
SET coluna1 = valor1, coluna2 = valor2, ...  
WHERE condição;
```

Exemplo:



```
UPDATE Clientes SET email = 'novo@email.com'  
WHERE id = 1;
```

DELETE

O comando DELETE é usado para remover registros de uma tabela.

Sintaxe:



```
DELETE FROM NomeDaTabela WHERE condição;
```

Exemplo:



```
DELETE FROM Clientes WHERE id = 3;
```

Gerenciamento de transações

Transações são um conjunto de operações que devem ser executadas como uma unidade única. O gerenciamento de transações garante que todas as operações sejam concluídas com sucesso ou que nenhuma alteração seja feita no banco de dados.

COMMIT

O comando COMMIT salva todas as alterações feitas durante a transação.



```
START TRANSACTION;  
UPDATE Produtos SET preco = preco * 0.90  
WHERE categoria = 'Promoção';  
DELETE FROM Produtos WHERE estoque = 0;  
COMMIT;
```

ROLLBACK

O comando ROLLBACK desfaz todas as alterações feitas durante a transação, retornando o banco de dados ao estado anterior ao início da transação.



```
START TRANSACTION;  
UPDATE Produtos SET preco = preco * 0.90  
WHERE categoria = 'Promoção';  
DELETE FROM Produtos WHERE estoque = 0;  
ROLLBACK;
```

Consultas e recuperação de dados

Comando SELECT

O comando SELECT é usado para consultar e recuperar dados de uma tabela. É a base das operações de leitura em SQL.

Sintaxe:



```
SELECT coluna1, coluna2, ... FROM NomeDaTabela;
```

Exemplo:



```
SELECT nome, email FROM Clientes;
```

Para selecionar todas as colunas, você pode usar o asterisco (*):



```
SELECT * FROM Clientes;
```

Cláusulas WHERE, ORDER BY, GROUP BY, HAVING

Estas cláusulas ajudam a refinar e organizar os dados retornados pelas consultas.

WHERE

A cláusula WHERE filtra os registros com base em uma condição específica.

Sintaxe:



```
SELECT * FROM Clientes WHERE cidade = 'Garça';
```

ORDER BY

A cláusula ORDER BY ordena os resultados com base em uma ou mais colunas.

Sintaxe:



```
SELECT * FROM Produtos ORDER BY preco DESC;
```

GROUP BY

A cláusula GROUP BY agrupa os registros que têm valores idênticos em colunas especificadas.

Sintaxe:



```
SELECT cidade, COUNT(*) FROM Clientes GROUP BY cidade;
```

HAVING

A cláusula HAVING é usada para filtrar grupos de registros após a aplicação da cláusula GROUP BY.

Sintaxe:



```
SELECT cidade, COUNT(*) FROM Clientes  
GROUP BY cidade HAVING COUNT(*) > 1;
```

Funções agregadas

As funções agregadas realizam cálculos em um conjunto de valores e retornam um único valor.

COUNT

Conta o número de registros.

Exemplo:



```
SELECT COUNT(*) FROM Clientes;
```

SUM

Soma os valores de uma coluna.

Exemplo:



```
SELECT SUM(preco) FROM Produtos;
```

AVG

Calcula a média dos valores de uma coluna.

Exemplo:



```
SELECT AVG(preco) FROM Produtos;
```

MAX

Retorna o valor máximo de uma coluna.

Exemplo:



```
SELECT MAX(preco) FROM Produtos;
```

MIN

Retorna o valor mínimo de uma coluna.

Exemplo:



```
SELECT MIN(preco) FROM Produtos;
```


Junções (JOINS)

As junções são usadas para combinar registros de duas ou mais tabelas com base em uma coluna relacionada entre elas.

INNER JOIN

Retorna registros que têm correspondências em ambas as tabelas.

Sintaxe:



```
SELECT Pedidos.id, Clientes.nome FROM Pedidos  
INNER JOIN Clientes ON Pedidos.cliente_id = Clientes.id;
```

LEFT JOIN

Retorna todos os registros da tabela da esquerda e os registros correspondentes da tabela da direita. Se não houver correspondência, os resultados da tabela da direita serão nulos.

Sintaxe:

```
SELECT Pedidos.id, Clientes.nome FROM Pedidos  
LEFT JOIN Clientes ON Pedidos.cliente_id = Clientes.id;
```

RIGHT JOIN

Retorna todos os registros da tabela da direita e os registros correspondentes da tabela da esquerda. Se não houver correspondência, os resultados da tabela da esquerda serão nulos.

Sintaxe:

```
SELECT Pedidos.id, Clientes.nome FROM Pedidos  
RIGHT JOIN Clientes ON Pedidos.cliente_id = Clientes.id;
```

FULL JOIN

Retorna registros quando há uma correspondência em uma das tabelas. Retorna todos os registros quando há uma correspondência em uma das tabelas.

Sintaxe:



```
SELECT Pedidos.id, Clientes.nome FROM Pedidos  
FULL OUTER JOIN Clientes  
ON Pedidos.cliente_id = Clientes.id;
```



MÓDULO 3

Node.js

Cada linha de código é invisível para o usuário, mas essencial para o mundo digital funcionar. O que você constrói hoje nos bastidores pode sustentar o futuro da tecnologia.

O que é

Node.js é um runtime de JavaScript construído no motor V8 do Google Chrome. Diferente dos navegadores, que executam JavaScript no lado do cliente, o Node.js permite que o JavaScript seja executado no lado do servidor. Isso significa que você pode usar a mesma linguagem tanto para a lógica de frontend quanto para a lógica de backend da sua aplicação.

Configuração e instalação

Vamos preparar seu ambiente de desenvolvimento para começar a criar aplicações backend incríveis.

O Node.js possui várias versões disponíveis, cada uma com suas próprias características e funcionalidades. Duas versões principais são amplamente utilizadas:

- LTS (Long-Term Support): Esta versão é recomendada para a maioria dos projetos de produção, pois recebe suporte a longo prazo e atualizações de segurança regulares.
- Current: Esta versão contém as funcionalidades mais recentes, mas não é recomendada para produção, pois pode não ser tão estável quanto a versão LTS.

Para instalar o Node.js, siga os passos de acordo com seu sistema operacional:

- Windows: Baixe o instalador do Node.js no site oficial e siga o assistente de instalação.
- macOS: Use o Homebrew ou baixe o instalador do Node.js no site oficial.
- Linux: Use o gerenciador de pacotes apropriado para sua distribuição, como o apt-get no Ubuntu.

Verificação da instalação

Após a instalação, você pode verificar se o Node.js e o npm (Node Package Manager) foram instalados corretamente. Abra o terminal e digite os seguintes comandos:



```
node -v  
npm -v
```

Esses comandos retornarão as versões instaladas do Node.js e npm.

Inicializando um projeto

Agora que você tem o Node.js, pode criar um novo projeto. Para isso, você pode usar os seguintes comandos:



```
npm init
```

Após esse comando irá aparecer algumas perguntas no seu terminal, você pode apenas teclar enter.

Isso irá criar um arquivo `package.json`, que é o arquivo de configuração do seu projeto.

Estrutura de pastas

Geralmente essa é a estrutura de pastas e arquivos de um projeto que usa Node:



```
meu-projeto/  
├─ node_modules/    (módulos e pacotes de terceiros)  
├─ src/              (seus arquivos JavaScript)  
├─ package.json      (arquivo de configuração o)  
├─ README.md         (documentação do projeto)
```

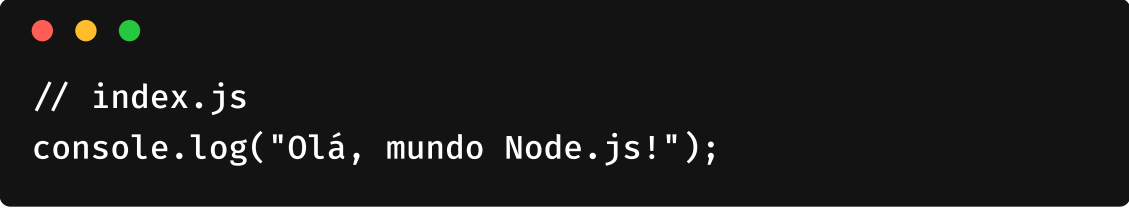
Agora que você configurou o ambiente, está pronto para começar a escrever código em Node.js.

Primeiros passos com Node.js

Node.js permite executar JavaScript fora do navegador. Isso abre um novo mundo de possibilidades, como criar servidores web, interagir com sistemas de arquivos, e muito mais.

Criando seu primeiro script

1. Crie um arquivo JavaScript: No seu projeto Node.js, crie um arquivo chamado `index.js`.
2. Escreva algum código JavaScript: Por exemplo, um simples `console.log`.



```
// index.js
console.log("Olá, mundo Node.js!");
```

Este código, embora simples, é poderoso. Ele mostra o básico da execução de um script Node.js.

Executando o programa

Para rodar seu programa, você precisa de um terminal ou linha de comando, por isso, recomendo usar o Visual Studio Code, pois ele tem um terminal incluso, na barra superior é só clicar em terminal.

1. Abra o terminal.
2. Execute o Script: Digite `node index.js` e pressione Enter.
3. Veja o resultado: Se tudo estiver correto, você verá "Olá, mundo Node.js!" impresso no terminal.

O que acontece aqui?

- node: É o comando para executar o Node.js.
- index.js: Indica ao Node.js qual arquivo executar.
- Resultado: O Node.js lê o arquivo `index.js`, executa o código JavaScript nele e imprime o resultado no terminal.
- Node.js como runtime: Ao contrário do JavaScript no navegador, onde seu código é executado dentro do contexto de uma página web, Node.js executa seu código como um processo independente em seu computador.

Fundamentos do Node.js

Vamos abordar alguns fundamentos do Node.js fique tranquilo caso você ainda não entenda alguns trechos como 'fs.readFile', ainda irei explicar durante esse guia. Mas caso você esteja TOTALMENTE perdido no código e não entenda a lógica dele o mais recomendado é focar os estudos em JavaScript antes do Node.

O que é o V8 Engine?

O V8 é um motor de execução de JavaScript open-source desenvolvido pelo Google. Ele foi projetado para o navegador Google Chrome, mas também é usado no Node.js para executar JavaScript no lado do servidor. O V8 compila o JavaScript diretamente para código de máquina nativo antes de executá-lo, o que resulta em uma performance extremamente rápida.

Event Loop e Assíncronismo

O Event Loop é o coração do Node.js e a chave para seu comportamento assíncrono. Diferente de linguagens de programação tradicionais que utilizam threads para lidar com múltiplas tarefas simultâneas, o Node.js usa um único thread, mas com um modelo de I/O não bloqueante e orientado a eventos.

Callbacks

Callbacks são funções passadas como argumento para outras funções e são chamadas quando a operação assíncrona é concluída.



```
const fs = require("fs");

fs.readFile("exemplo.txt", "utf8", (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

O código acima basicamente leu o conteúdo que está dentro do arquivo exemplo.txt e mostrou no terminal.

Caso queira fazer o teste, lembre-se de deixar no mesmo diretório do seu arquivo index.js

Problema do Callback Hell

O uso extensivo de callbacks pode levar ao "callback hell", onde o código fica aninhado em vários níveis, tornando-se difícil de ler e manter.

Imagine que queremos ler um arquivo e, a partir disso, realizar várias operações encadeadas.

```
fs.readFile("exemplo.txt", "utf8", (err, data) => {
  if (err) {
    console.error(err);
  } else {
    console.log("Primeira leitura:", data);

    fs.readFile("outro.txt", "utf8", (err, data2) => {
      if (err) {
        console.error(err);
      } else {
        console.log("Segunda leitura:", data2);

        fs.readFile("mais_um.txt", "utf8", (err, data3) => {
          if (err) {
            console.error(err);
          } else {
            console.log("Terceira leitura:", data3);
          }
        });
      }
    });
  }
});
// ... e assim por diante
```

Promises

Promises são uma forma mais moderna e legível de lidar com operações assíncronas. Elas representam um valor que pode estar disponível agora, no futuro ou nunca.



```
const fs = require("fs").promises;

fs.readFile("exemplo.txt", "utf8")
  .then((data) => {
    console.log(data);
  })
  .catch((err) => {
    console.error(err);
  });
```

Async/Await

Async/await é uma sintaxe que torna o uso de promises mais simples e legível, permitindo escrever código assíncrono de forma semelhante ao código síncrono.



```
const fs = require("fs").promises;

async function readFile() {
  try {
    const data = await fs.readFile("teste.txt", "utf8");
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

readFile();
```

Módulos e pacotes

Sistema de módulos do Node.js

O Node.js utiliza um sistema de módulos para organizar e estruturar o código. Módulos são arquivos JavaScript que exportam funcionalidades para serem utilizadas em outros arquivos. Existem dois tipos principais de módulos no Node.js:

- Módulos internos: Incluídos no núcleo do Node.js, como fs (sistema de arquivos), http (servidor HTTP), e path (manipulação de caminhos de arquivos).
- Módulos externos: Instalados via npm (Node Package Manager) e gerenciados como dependências do projeto.

Importação e exportação de módulos

Para utilizar um módulo, você deve importá-lo usando a função `require()`. Para tornar funções ou variáveis disponíveis para outros arquivos, você deve exportá-las.

Módulo interno



```
const fs = require("fs");
```

O `require` é a forma clássica do Node.js de importar módulos (pacotes de funcionalidades).

O `"fs"` é um módulo nativo do Node.js chamado File System (sistema de arquivos).

Ele permite ler, escrever, criar e deletar arquivos e pastas no seu computador.

Na constante fs guardamos o módulo.

Assim, sempre que você usar fs, estará acessando funções para manipular arquivos.

Módulo externo

Crie um arquivo chamado math.js com o seguinte conteúdo:



```
// math.js
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

module.exports = { add, subtract };
```


Importe e utilize o módulo em outro arquivo:



```
// index.js
const math = require("./math");

console.log(math.add(2, 3)); // 5
console.log(math.subtract(5, 3)); // 2
```

O que estamos fazendo aqui?

Em vez de escrever todas as funções dentro de um único arquivo, criamos um módulo externo chamado math.js.

Dentro dele, declaramos duas funções: add e subtract.

No final usamos:



```
module.exports = { add, subtract };
```

Isso significa que estamos exportando essas funções, para que possam ser utilizadas em outros arquivos.

No index.js, usamos:

```
const math = require("./math");
```

Aqui, o `require("./math")` importa tudo o que foi exportado do arquivo `math.js`.

Como exportamos um objeto `{ add, subtract }`, o `math` recebe esse objeto.

Usando o módulo

Agora podemos acessar as funções como propriedades do objeto `math`:

```
math.add(2, 3); // retorna 5  
math.subtract(5, 3); // retorna 2
```

Resumindo:

math.js - define funções e as exporta com module.exports.

index.js - importa o módulo com require("./math") e utiliza as funções.

Leitura de arquivos

A leitura de arquivos pode ser feita de forma síncrona ou assíncrona. No Node.js, é recomendado usar operações assíncronas para evitar o bloqueio do Event Loop.

Leitura assíncrona



```
const fs = require("fs");

fs.readFile("exemplo.txt", "utf8", (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

Leitura síncrona



```
const fs = require("fs");

try {
  const data = fs.readFileSync("exemplo.txt", "utf8");
  console.log(data);
} catch (err) {
  console.error(err);
}
```

Escrita de arquivos

Assim como na leitura, a escrita de arquivos pode ser realizada de forma síncrona ou assíncrona.

Escrita assíncrona



```
const fs = require("fs");

const content = "Algum conteúdo!";

...
```

...

```
fs.writeFile("exemplo.txt", content, (err) => {  
  if (err) {  
    console.error(err);  
    return;  
  }  
  console.log("O arquivo foi escrito");  
});
```

Escrita síncrona

```
const fs = require("fs");  
  
const content = "Algum conteúdo!";  
  
try {  
  fs.writeFileSync("exemplo.txt", content);  
  console.log("O arquivo foi escrito");  
} catch (err) {  
  console.error(err);  
}
```

Faça o teste você mesmo. Rode o código e depois acesse o arquivo .txt para visualizar.

Manipulação de diretórios

Você pode criar novos diretórios usando `fs.mkdir`.



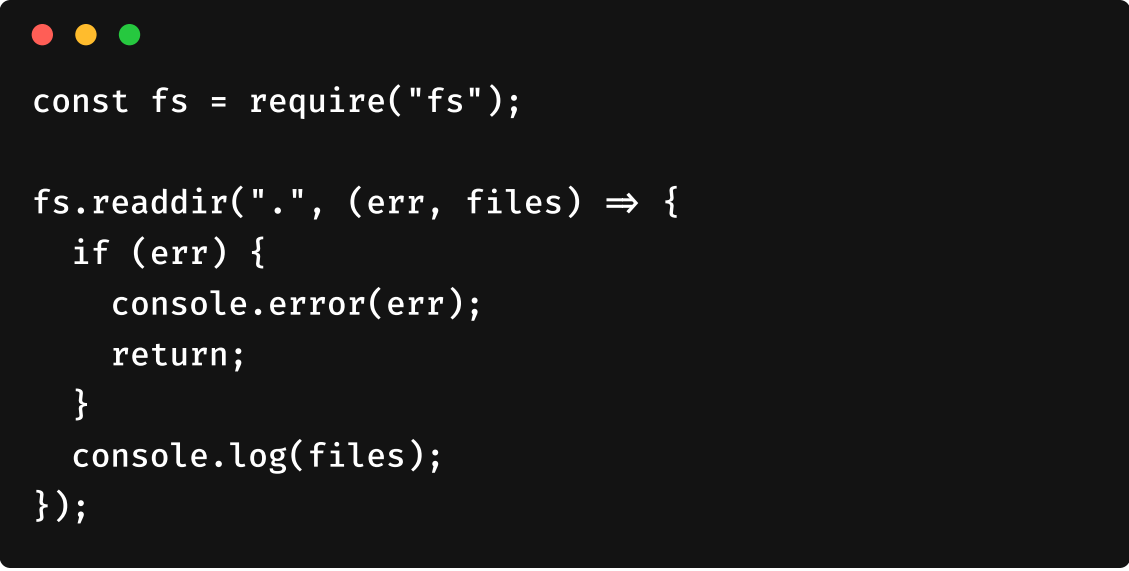
```
const fs = require("fs");

fs.mkdir("novo-local", { recursive: true }, (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log("Diretórios criado");
});
```

Se você entrar no diretório agora irá ver uma nova pasta chamada 'novo-local'.

Leitura de diretórios

Para ler o conteúdo de um diretório, use `fs.readdir`.



```
const fs = require("fs");

fs.readdir(".", (err, files) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(files);
});
```

O método `fs.readdir` do Node.js é usado para ler o conteúdo de um diretório (pasta).

Ele retorna uma lista de arquivos e subpastas que estão dentro do diretório especificado.

Streams

Streams são uma forma eficiente de ler e escrever grandes quantidades de dados. Em vez de carregar o arquivo inteiro na memória, você pode processá-lo em partes.



```
const fs = require("fs");

const readStream = fs.createReadStream(
  "exemplo.txt", "utf8"
);

readStream.on("data", (chunk) => {
  console.log(chunk);
});

readStream.on("end", () => {
  console.log("Leitura concluída");
});

readStream.on("error", (err) => {
  console.error(err);
});
```

O `fs.createReadStream` é um método do módulo nativo `fs` (File System) do Node.js usado para criar um fluxo de leitura (readable stream) a partir de um arquivo.

Em vez de ler o arquivo inteiro de uma vez (como o `fs.readFile` faz), ele vai lendo o arquivo em pedaços (chunks), enviando esses dados aos poucos.

Escrita de arquivos com streams

```
const fs = require("fs");

const writeStream = fs.createWriteStream("grande.txt");

writeStream.write("Alguns dados");
writeStream.write("Mais alguns dados");

writeStream.end();

writeStream.on("finish", () => {
  console.log("Escrita finalizada");
});

writeStream.on("error", (err) => {
  console.error(err);
});
```

Trabalhando com HTTP

O Node.js possui um módulo interno chamado `http` que permite criar servidores web e lidar com requisições e respostas HTTP. Este módulo fornece a base para a construção de aplicações web e APIs.

Introdução ao HTTP

HTTP (HyperText Transfer Protocol) é o protocolo fundamental usado na web para a comunicação entre clientes e servidores. Ele define como as mensagens são formatadas e transmitidas, e quais ações os servidores e navegadores devem tomar em resposta a diferentes comandos.

Componentes de uma requisição HTTP

1. Método HTTP: Define a ação a ser realizada. Os métodos mais comuns incluem:

- GET: Solicita dados de um servidor.
- POST: Envia dados para um servidor para criar ou atualizar recursos.
- PUT: Atualiza um recurso existente no servidor.
- DELETE: Remove um recurso do servidor.

2. URL (Uniform Resource Locator): Especifica o endereço do recurso a ser acessado.

3. Headers: Contêm informações adicionais sobre a requisição, como tipo de conteúdo, autenticação, etc.

4. Body: Usado para enviar dados ao servidor (geralmente em requisições POST e PUT).

Componentes de uma resposta HTTP

1. Status Code: Indica o resultado da requisição. Alguns códigos comuns:

- 200 OK: A requisição foi bem-sucedida.
- 404 Not Found: O recurso solicitado não foi encontrado.
- 500 Internal Server Error: Ocorreu um erro no servidor.

2. Headers: Contêm informações sobre a resposta, como tipo de conteúdo, tamanho, etc.

3. Body: Contém os dados retornados pelo servidor, como HTML, JSON.

Criando um servidor HTTP básico

Para criar um servidor HTTP básico, você precisa utilizar o módulo `http` e definir como o servidor deve responder às requisições.

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain");
  res.end("Hello World!");
});

...
```



```
const PORT = 3000;
server.listen(PORT, () => {
  console.log(
    `Server running at http://localhost:${PORT}`
  );
});
```

Depois de executar o comando `node index.js`, você pode abrir seu navegador e acessar `http://localhost:3000` para ver a mensagem "Hello World!" exibida.

Tratamento de requisições e respostas

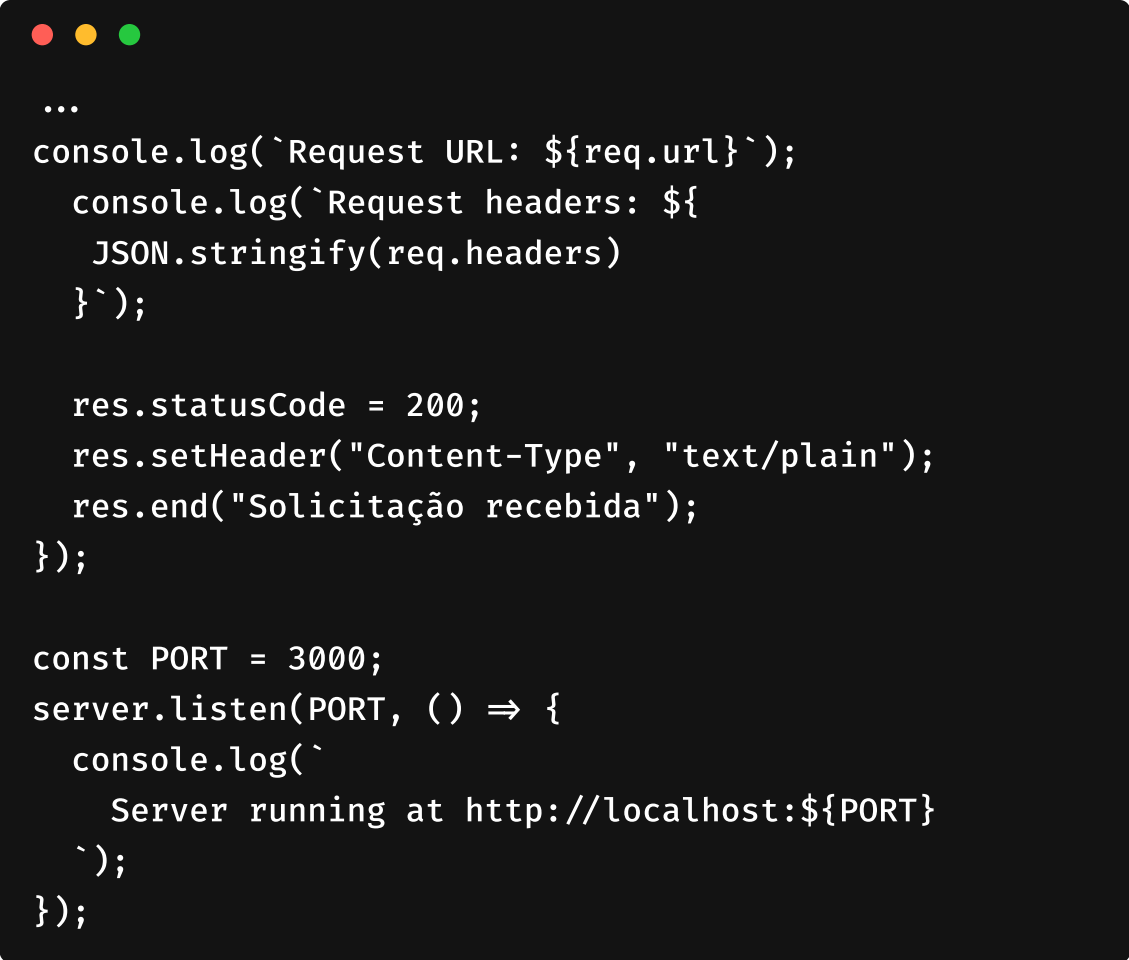
Requisições (Requests)

As requisições HTTP possuem várias propriedades que podem ser acessadas para obter informações sobre a solicitação.



```
const http = require("http");

const server = http.createServer((req, res) => {
  console.log(`Request method: ${req.method}`);
  ...
});
```



```
...
console.log(`Request URL: ${req.url}`);
console.log(`Request headers: ${
  JSON.stringify(req.headers)
}`);

res.statusCode = 200;
res.setHeader("Content-Type", "text/plain");
res.end("Solicitação recebida");
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`
    Server running at http://localhost:${PORT}
  `);
});
```

Respostas (Responses)

Para enviar uma resposta ao cliente, você pode definir o status, os headers e o corpo da resposta.



```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/plain") {
    res.statusCode = 200;
    res.setHeader("Content-Type", "text/plain");
    res.end("Hello, plain text!");
  } else if (req.url === "/json") {
    res.statusCode = 200;
    res.setHeader("Content-Type", "application/json");
    res.end(JSON.stringify({ message: "Hello, JSON!" }));
  } else {
    res.statusCode = 404;
    res.setHeader("Content-Type", "text/plain");
    res.end("Not Found");
  }
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`
    Server running at http://localhost:${PORT}
  `);
});
```

Após executar o código, abra o navegador e digite:

`http://localhost:3000/plain` - Deve aparecer: Hello, plain text!

`http://localhost:3000/json` → Deve aparecer: `{"message": "Hello, JSON!"}`

`http://localhost:3000/qualquercoisa` - Deve aparecer: Not Found

Introdução ao Express

Express é um framework para Node.js que simplifica o desenvolvimento de aplicações web e APIs. Ele fornece uma série de recursos robustos para criar servidores e gerenciar rotas, middlewares, e muito mais.

Instalação do Express

Para começar a usar o Express, você precisa instalá-lo em seu projeto Node.js. Use o npm para instalar o pacote:



```
npm install express
```

Criando um servidor Express

No arquivo `index.js` adicione o seguinte código para configurar um servidor básico:



```
const express = require("express");
const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("Hello, Express!");
});

app.listen(PORT, () => {
  console.log(`
    Server is running on http://localhost:${PORT}
  `);
});
```

Para iniciar o servidor, execute o comando:



```
node index.js
```

Agora abra seu navegador e acesse <http://localhost:3000>. Você deverá ver a mensagem: "Hello, Express!".

Roteamento avançado

O roteamento no Express é uma forma de determinar como uma aplicação responde a uma requisição de um cliente em um endpoint específico.

Definindo rotas

Você pode definir rotas para diferentes métodos HTTP (GET, POST, PUT, DELETE, etc.) e caminhos.



```
const express = require("express");
const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("GET solicitação para a página inicial");
});

app.post("/submit", (req, res) => {
  res.send("POST solicitação para /submit");
});

...
```



```
app.put("/update", (req, res) => {
  res.send("PUT solicitação para /update");
});

app.delete("/delete", (req, res) => {
  res.send("DELETE solicitação para /delete");
});

app.listen(PORT, () => {
  console.log(`
    Server is running on http://localhost:${PORT}
  `);
});
```

Tratamento de erros

O tratamento de erros no Express é feito através de middlewares de erro. Um middleware de erro tem quatro argumentos: `err`, `req`, `res`, e `next`.

Exemplo de middleware de erro:



```
const express = require("express");
const app = express();
const PORT = 3000;
...
```



```
// Middleware de tratamento de erros
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Algo quebrou!");
};

app.get("/", (req, res) => {
  throw new Error("Quebrou");
});

app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`
    Server is running on http://localhost:${PORT}
  `);
});
```

Banco de dados

Conectar o Node.js a bancos de dados é uma tarefa comum no desenvolvimento de aplicações web. Neste tópico, abordaremos a conexão com bancos de dados SQL usando ORM (Object-Relational Mapping) e a configuração de um banco de dados utilizando Docker.

Conectando Node.js a bancos de dados SQL

Para se conectar a bancos de dados SQL como MySQL, utilizaremos o Sequelize, um ORM popular para Node.js que facilita a interação com bancos de dados.

Instalando o Sequelize



```
npm install sequelize
```

Instalar o MySQL



```
npm install mysql2
```

Configurando o Sequelize

Essa é a nossa configuração básica para configurar a conexão com o banco de dados.



```
const { Sequelize } = require("sequelize");

// Lembre-se de troca para o nome do seu banco ...
// ... de dados usuário e senha.
const sequelize = new Sequelize(
  "database", "username", "password", {
    host: "localhost",
    dialect: "mysql",
  });

async function connectDB() {
  try {
    await sequelize.authenticate();
    console.log("Conexão estabelecida com sucesso!");
  } catch (error) {
    console.error("Erro ao conectar:", error);
  }
}

connectDB();

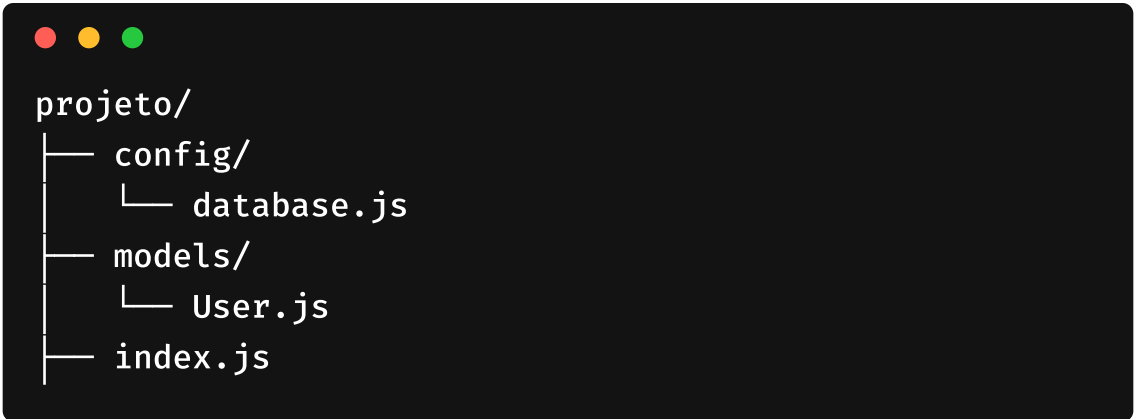
module.exports = sequelize;
```

Lembrando: Ainda teremos um guia ensinando banco de dados com MySQL.

Definindo modelos e executando operações

Os modelos no Sequelize representam tabelas no banco de dados. Você pode definir modelos para criar, ler, atualizar e excluir dados.

Vamos ter a seguinte estrutura de pasta:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays a directory tree for a project named 'projeto'.

```
projeto/  
├── config/  
│   └── database.js  
├── models/  
│   └── User.js  
└── index.js
```

Cada parte do projeto tem sua responsabilidade:

config/database.js - Apenas configuração do Sequelize.

models/user.js - Definição do model.

index.js - Inicialização do app e execução.



```
// index.js
const sequelize = require("../config/database");
const User = require("../models/user");

// Testar conexão
async function connectDB() {
  try {
    await sequelize.authenticate();
    console.log("✅ Conexão estabelecida com sucesso!");
  } catch (error) {
    console.error("❌ Erro ao conectar:", error);
  }
}

// Criar tabelas e inserir exemplo
async function syncDatabase() {
  try {
    await sequelize.sync({ force: true });
    // ⚠️ recria tabelas a cada execução
    console.log("✅ Database & tabelas criados!");
    ...
  }
}
```

```

// Criar usuário de teste
const newUser = await User.create({
  username: "iuri",
  email: "teste@email.com",
});
console.log("👤 Usuário criado:", newUser.toJSON());
} catch (error) {
  console.error(
    "❌ Erro ao sincronizar banco:", error
  );
}
}

// Executa as funções
(async () => {
  await connectDB();
  await syncDatabase();
})();
```

```

// config/database.js
const { Sequelize } = require("sequelize");

// Troque "database", "username" e "password" ...
// ...pelos seus dados do MySQL
...
```




```
const sequelize = new Sequelize(
  "database", "username", "password", {
    host: "localhost",
    dialect: "mysql",
  });

module.exports = sequelize;
```



```
// models/user.js
const { DataTypes } = require("sequelize");
const sequelize = require("../config/database");

const User = sequelize.define("User", {
  username: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
  },
});

module.exports = User;
```

Realizando operações CRUD

Criando registro

```
const sequelize = require("../config/database");
const User = require("../models/User");

// Criar usuário
async function createUser() {
  try {
    const user = await User.create({
      username: "john_doe",
      email: "john@example.com",
    });
    console.log("✅ Usuário criado:", user.toJSON());
  } catch (error) {
    console.error("❌ Erro ao criar usuário:", error);
  }
}

async function main() {
  try {
    // Conecta e sincroniza banco
    await sequelize.authenticate();
    console.log("✅ Conexão estabelecida com sucesso!");
  }
  ...
}
```

```
...
    await sequelize.sync();

    // Executa operação CRUD
    await createUser();
  } catch (error) {
    console.error("❌ Erro no processo:", error);
  } finally {
    await sequelize.close();
    // fecha conexão após execução
  }
}

main();
```

Lendo registros

```
const User = require("../models/user");

async function getUsers() {
  try {
    const users = await User.findAll();
    console.log("Users:", users);
  }
  ...
}
```

```
...
    } catch (error) {
      console.error("Error fetching users:", error);
    }
  }

  getUsers();
```

Atualizando registros

```
const User = require("../models/user");

async function updateUser() {
  try {
    const user = await User.findOne({ where:
      { username: "john_doe" }
    });
    if (user) {
      user.email = "new_email@example.com";
      await user.save();
      console.log("User updated:", user);
    }
  }
  ...
}
```

```
...
    } catch (error) {
      console.error("Error updating user:", error);
    }
  }

  updateUser();
```

Excluindo registros

```
const User = require("../models/user");

async function deleteUser() {
  try {
    const user = await User.findOne({
      where: { username: "john_doe" }
    });
    if (user) {
      await user.destroy();
      console.log("User deleted");
    }
  }
  ...
}
```

```
...  
    } catch (error) {  
        console.error("Error deleting user:", error);  
    }  
}  
  
deleteUser();
```

Autenticação

Importante: Neste tópico é um pouco mais complexo de entender, ainda iremos abordar melhor sobre JWT e Postman. Objetivo aqui é apenas abordar que podemos fazer autenticação com Node.js

A autenticação e a autorização são partes fundamentais de qualquer aplicação web, garantindo que apenas usuários autorizados possam acessar certos recursos ou executar determinadas ações. Neste tópico, vamos focar na autenticação básica usando JWT (JSON Web Tokens).

Introdução à autenticação com JWT

JWT é um padrão aberto (RFC 7519) que define uma maneira compacta e segura de transmitir informações entre as partes como um objeto JSON. Essas informações podem ser verificadas e confiadas porque são assinadas digitalmente.

Componentes de um JWT

Um JWT é composto por três partes separadas por pontos (`. `):

1. Header Contém o tipo de token e o algoritmo de assinatura.
2. Payload: Contém as declarações (claims). São as informações que você quer transmitir.
3. Signature: É a assinatura gerada a partir do header, payload e uma chave secreta.

Exemplo de um JWT:



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJpdXJpIiwiaWZw1haWwiOiJ0ZXN0ZUBlbWFpbC5jb20iLCJpYXQiOiJlE3NTY5ODg2OTksImV4cCI6MTc1Njk5MjI5OX0.OMc10hTGKyuu7AdJmmZ08b1Xj1bViTY5x\_-4xpM-Afk
```

Gerando Tokens



```
const express = require("express");
const jwt = require("jsonwebtoken");
const mysql = require("mysql2/promise");
// versão com async/await
...
```



```
const app = express();
app.use(express.json());

const SECRET_KEY = "minha_chave_secreta";
// em produção use variável de ambiente

// Conexão com banco
const db = mysql.createPool({
  host: "localhost",
  user: "username",
  password: "password",
  database: "database",
});

// Função para gerar token
function generateToken(user) {
  return jwt.sign(
    { id: user.id, username: user.username,
      email: user.email },
    SECRET_KEY,
    { expiresIn: "1h" }
  );
}
...
```




```
// Middleware para verificar token
function verifyToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];

  if (!token) {
    return res.status(401).json(
      { message: "Token não fornecido" }
    );
  }

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) {
      return res.status(403).json(
        { message: "Token inválido ou expirado" }
      );
    }
    req.user = user;
    next();
  });
}

// Rota de login (consulta banco de dados)
app.post("/login", async (req, res) => {
  console.log("REQ BODY:", req.body);
  const { email } = req.body;
  ...
```



```
...
try {
  const [rows] = await db.query(
    "SELECT * FROM users WHERE email = ?", [
      email,
    ]
  );

  if (rows.length === 0) {
    return res.status(401).json(
      { message: "Usuário não encontrado" }
    );
  }

  const user = rows[0];

  const token = generateToken(user);

  return res.json({ token });
} catch (err) {
  console.error(err);
  res.status(500).json(
    { message: "Erro no servidor" }
  );
}
});
```



```
// Rota protegida
app.get("/dashboard", verifyToken, (req, res) => {
  res.json({ message: `Bem-vindo, ${req.user.username}!`,
    user: req.user });
});

app.listen(3000, () => {
  console.log("Rodando em http://localhost:3000");
});
```

Esse código cria um sistema básico de login com JWT + rota protegida, onde:

1. O usuário faz login enviando apenas o email.
2. Se existir no banco recebe um token JWT.
3. Com esse token, consegue acessar rotas protegidas (/dashboard).



MÓDULO 4

PHP

Cada linha de código é invisível para o usuário, mas essencial para o mundo digital funcionar. O que você constrói hoje nos bastidores pode sustentar o futuro da tecnologia.

O que é

O PHP (Hypertext Preprocessor) é uma linguagem de programação voltada para o desenvolvimento web, criada originalmente em 1994 por Rasmus Lerdorf. Ela é usada principalmente no lado do servidor (backend), ou seja, o código PHP é executado no servidor e o resultado (geralmente em HTML) é enviado para o navegador do usuário.

Diferente de linguagens como JavaScript, que rodam diretamente no navegador, o PHP trabalha por trás das cortinas, processando dados, acessando bancos de dados e gerando páginas dinâmicas. É como o cérebro por trás de um site, fazendo tudo funcionar de forma inteligente e automatizada.

O PHP está presente em boa parte da internet, estima-se que mais de 70% dos sites que usam linguagens de servidor dependem dele. Plataformas famosas como WordPress, Facebook (em seus primórdios), Wikipedia, Magento e Laravel usam (ou usaram) PHP como base.

O que você precisa ter instalado

Antes de começar a escrever suas primeiras linhas de PHP, é importante preparar o seu ambiente de desenvolvimento. Isso significa instalar os programas necessários para que o PHP funcione corretamente no seu computador. Vamos ao passo a passo!

Instale o XAMPP

O XAMPP é um pacote gratuito que traz tudo o que você precisa para rodar PHP localmente:

Apache → o servidor web que executa o PHP.

MySQL → o banco de dados para armazenar informações.

PHP → a linguagem em si.

phpMyAdmin → uma ferramenta visual para gerenciar bancos de dados.

Passos para instalar:

Acesse o site oficial: https://www.apachefriends.org/pt_br/index.html

Baixe a versão compatível com o seu sistema operacional (Windows, macOS ou Linux).

Siga o instalador clicando em Next > Next > Finish.

Abra o XAMPP Control Panel e clique em Start nos módulos Apache e MySQL.

Para testar, abra o navegador e digite: <http://localhost>

Se aparecer a tela do XAMPP, tudo está funcionando!

Baixe um editor de código

Você precisará de um bom editor para escrever seus códigos PHP. O mais popular é o Visual Studio Code (VS Code), da Microsoft, ele é leve, gratuito e cheio de extensões úteis.

Passos para instalar:

Baixe em: <https://code.visualstudio.com/>

Instale normalmente.

(Opcional) Instale algumas extensões úteis:

PHP Intelephense – Autocompletar e dicas de código.

PHP Server – Para rodar pequenos projetos direto do VS Code.

Material Icon Theme – Ícones bonitos para pastas e arquivos.

Crie a pasta do seu projeto

No seu computador, abra o diretório onde o XAMPP está instalado.

Normalmente: `C:\xampp\htdocs\`

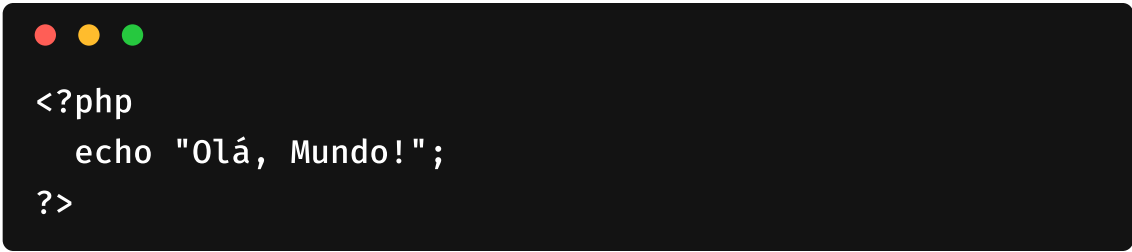
Dentro de `htdocs`, crie uma nova pasta com o nome do seu projeto, por exemplo: `C:\xampp\htdocs\projeto`

Teste seu primeiro arquivo PHP

Agora, vamos ver se tudo está funcionando:

Abra o VS Code e crie um arquivo chamado index.php

Dentro dele, digite:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains the following PHP code:

```
<?php
    echo "Olá, Mundo!";
?>
```

```
<?php
    echo "Olá, Mundo!";
?>
```

Salve o arquivo e abra no navegador:

<http://localhost/projetinho>

Se aparecer 'Olá, Mundo!', parabéns! Você acabou de rodar seu primeiro código PHP.

Fundamentos do PHP

Agora que seu ambiente está pronto, é hora de mergulhar nos fundamentos do PHP, ou seja, a base da linguagem. Esses conceitos são essenciais para qualquer desenvolvedor, pois estão presentes em todo tipo de projeto de sites simples até sistemas completos!

Variáveis e tipos de dados

As variáveis são como “caixinhas” onde você guarda informações temporárias enquanto o código roda.

Em PHP, toda variável começa com o símbolo \$.

Exemplo:

```
<?php
$nome = "Iuri";
$idade = 25;
$altura = 1.87;
$estudando = true;

echo "Meu nome é $nome e eu tenho $idade anos.";
?>
```

Principais tipos de dados:

String → texto ("Olá, mundo")

Integer → número inteiro (25)

Float → número decimal (1.75)

Boolean → verdadeiro ou falso (true / false)

Array → conjunto de dados (["PHP", "HTML", "CSS"])

Object → estruturas mais complexas (usadas em POO)

Operadores aritméticos e lógicos

Os operadores aritméticos são usados para fazer cálculos matemáticos:



Operador	Função	Exemplo	Resultado
<code>+</code>	Soma	<code>\$x + \$y</code>	7
<code>-</code>	Subtração	<code>\$x - \$y</code>	3
<code>*</code>	Multiplicação	<code>\$x * \$y</code>	10
<code>/</code>	Divisão	<code>\$x / \$y</code>	2
<code>%</code>	Módulo (resto)	<code>\$x % \$y</code>	1

Exemplo prático:



```
<?php
$x = 10;
$y = 3;

echo $x + $y; // 13
echo $x % $y; // 1
?>
```

Os operadores lógicos são usados em comparações e condições:

Operador	Significado	Exemplo
<code>`=`</code>	Igualdade	<code>`\$a = \$b`</code>
<code>`≠`</code>	Diferente	<code>`\$a ≠ \$b`</code>
<code>`>`</code>	Maior que	<code>`\$a > \$b`</code>
<code>`<`</code>	Menor que	<code>`\$a < \$b`</code>
<code>`&&`</code>	E lógico	<code>`\$a > 0 && \$b > 0`</code>

Estruturas de decisão (if, else, switch)

Essas estruturas permitem que o programa tome decisões com base em condições. Exemplo com if e else:

```
<?php
$idade = 18;

if ($idade ≥ 18) {
    echo "Você é maior de idade.";
} else {
    echo "Você é menor de idade.";
}
?>
```

Exemplo com elseif:



```
<?php
$nota = 8;

if ($nota ≥ 9) {
    echo "Excelente!";
} elseif ($nota ≥ 6) {
    echo "Aprovado!";
} else {
    echo "Reprovado!";
}
?>
```

Exemplo com switch:



```
<?php
$dia = "sexta";

switch ($dia) {
    case "segunda":
        echo "Início da semana!";
        break;
    ...
}
```

```
...
case "sexta":
    echo "Sextou!";
    break;
default:
    echo "Dia normal ... ";
}
?>
```

Laços (for, while, foreach)

Os laços (ou loops) são usados para repetir ações várias vezes — perfeitos para percorrer listas, gerar conteúdo dinâmico e automatizar tarefas.

for — quando você sabe quantas vezes repetir

```
<?php
for ($i = 1; $i ≤ 5; $i++) {
    echo "Número: $i <br>";
}
?>
```

while — repete enquanto a condição for verdadeira



```
<?php
$contador = 1;

while ($contador ≤ 3) {
    echo "Contando: $contador <br>";
    $contador++;
}
?>
```

foreach — ideal para percorrer arrays



```
<?php
$linguagens = ["PHP", "JavaScript", "Python"];

foreach ($linguagens as $linguagem) {
    echo "Estou aprendendo $linguagem <br>";
}
?>
```

Funções e Arrays

Agora que você já entende variáveis, operadores e estruturas de repetição, chegou a hora de conhecer um dos pilares do PHP: as funções e os arrays.

Esses dois conceitos são essenciais para organizar seu código, evitar repetições e trabalhar com conjuntos de dados de forma inteligente.

Criando funções

As funções são blocos de código criados para executar uma tarefa específica. Elas tornam o código mais limpo, reutilizável e fácil de manter.



```
function nomeDaFuncao() {  
    // Código a ser executado  
}
```

Exemplo prático:

```
<?php
function saudacao() {
    echo "Olá, bem-vindo ao curso de PHP!";
}

saudacao(); // Chamando a função
?>
```

Você também pode passar parâmetros para funções, tornando-as mais dinâmicas:

```
<?php
function somar($a, $b) {
    return $a + $b;
}

$resultado = somar(10, 5);
echo "O resultado da soma é: $resultado";
?>
```


Escopo de variáveis

O escopo define onde uma variável pode ser acessada dentro do código.

Variável local: só existe dentro da função.

Variável global: pode ser acessada em qualquer parte do código, mas precisa ser declarada como global dentro de uma função.



```
<?php
$mensagem = "Olá do escopo global!";

function mostrarMensagem() {
    global $mensagem; // permite usar variável global
    echo $mensagem;
}

mostrarMensagem();
?>
```

Dica: use variáveis globais com moderação. Prefira passar valores por parâmetro — é mais seguro e fácil de manter.

Arrays associativos e multidimensionais

Um array é uma estrutura que guarda vários valores em uma única variável. Existem diferentes tipos de arrays no PHP:

Array numérico (índices automáticos)

```
<?php
$frutas = ["Maçã", "Banana", "Laranja"];
echo $frutas[1]; // Saída: Banana
?>
```

Array associativo (índices personalizados)

Cada elemento é identificado por um nome (chave), em vez de um número.

```
<?php
$pessoa = [
    "nome" => "Iuri",
    "idade" => 25,
    ...
];
```

```
...  
"profissao" => "Desenvolvedor"  
];  
  
echo $pessoa["nome"]; // Saída: Iuri  
?>
```

Array multidimensional

É um array dentro de outro — muito útil para representar tabelas ou coleções de dados complexos.

```
<?php  
$usuarios = [  
    ["nome" => "Vitória", "idade" => 22],  
    ["nome" => "Iuri", "idade" => 25],  
    ["nome" => "Carlos", "idade" => 30]  
];  
  
echo $usuarios[1]["nome"]; // Saída: Iuri  
?>
```

Funções nativas úteis

O PHP possui centenas de funções prontas, que facilitam o trabalho do dia a dia. Aqui estão algumas das mais usadas:

Manipulação de strings



```
strlen("PHP"); // Retorna o tamanho da string (3)
strtoupper("php"); // Converte para MAIÚSCULAS
strtolower("PHP"); // Converte para minúsculas
str_replace("mundo", "Iuri"); // Substitui palavras
```

Funções matemáticas



```
abs(-10); // Retorna 10
pow(2, 3); // Potência → 8
rand(1, 100); // Número aleatório entre 1 e 100
```

Funções para arrays



```
count($frutas); // Conta quantos itens há no array
array_push($frutas, "Uva"); // Adiciona novo item
array_pop($frutas); // Remove o último item
in_array("Maçã", $frutas); // Verifica se existe
```

Exemplo completo:



```
<?php
function apresentarPessoa($dados) {
    echo "Nome: " . $dados["nome"] . "<br>";
    echo "Idade: " . $dados["idade"] . " anos<br>";
}

$pessoas = [
    ["nome" => "Iuri", "idade" => 25],
    ["nome" => "Vitória", "idade" => 22]
];

foreach ($pessoas as $pessoa) {
    apresentarPessoa($pessoa);
}
?>
```

O que acontece aqui:

Criamos uma função que mostra os dados de uma pessoa.

Guardamos várias pessoas dentro de um array multidimensional.

Usamos um foreach para percorrer todas e exibir seus dados.

Manipulação de formulários

Os formulários são a ponte entre o usuário e o servidor.

Sempre que alguém preenche um campo e clica em “Enviar”, os dados são enviados para o PHP processar — e é aí que entra a manipulação de formulários.

Vamos ver, passo a passo, como tudo funciona.

Método GET e POST

Existem duas formas principais de enviar dados de um formulário para o PHP: GET e POST.

Método GET

Envia os dados pela URL (visíveis no navegador).

Ideal para pesquisas, filtros e parâmetros simples.

Exemplo: `pagina.php?nome=luri&idade=25`



```
<form action="recebe.php" method="get">
  <label>Nome:</label>
  <input type="text" name="nome" />
  <button type="submit">Enviar</button>
</form>
```



```
// recebe.php
<?php
$nome = $_GET["nome"];
echo "Olá, $nome!";
?>
```

Método GET e POST

Resultado: Se o usuário digitar "luri", a URL ficará assim:

<http://localhost/recebe.php?nome=luri>

E o navegador exibirá:

Olá, luri!

Método POST

Envia os dados “escondidos”, dentro do corpo da requisição.

Mais seguro e indicado para formulários de login, cadastros e dados sensíveis.

```
<form action="processa.php" method="post">
  <label>Email:</label>
  <input type="email" name="email" />

  <label>Senha:</label>
  <input type="password" name="senha" />

  <button type="submit">Entrar</button>
</form>
```

```
// processa.php

<?php
$email = $_POST["email"];
$senha = $_POST["senha"];

echo "Email informado: $email";
?>
```


Use POST sempre que os dados forem privados ou alterarem o sistema.

Validação de dados

A validação serve para garantir que o usuário preencheu o formulário corretamente, antes de salvar no banco de dados ou continuar o processo.

Exemplo simples:

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $nome = $_POST["nome"];
    $email = $_POST["email"];

    if (empty($nome) || empty($email)) {
        echo "Por favor, preencha todos os campos!";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "E-mail inválido!";
    } else {
        echo "Dados válidos! Obrigado, $nome.";
    }
}
?>
```

O que está acontecendo:

Verificamos se o formulário foi enviado via POST.

Checamos se os campos estão vazios.

Validamos o formato do e-mail com `filter_var()`.

Sanitização e segurança básica

Além de validar, é essencial limpar os dados antes de usá-los — isso evita ataques como XSS (Cross-Site Scripting) e injeção de código.

Exemplo com sanitização:

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Remove espaços, tags HTML e caracteres perigosos
    $nome = trim($_POST["nome"]);
    $nome = htmlspecialchars($nome);

    $email = filter_var($_POST["email"],
        FILTER_SANITIZE_EMAIL);

    echo "Olá, $nome! Seu e-mail ($email) foi recebido";
}
?>
```

Boas práticas de segurança:

Use htmlspecialchars() para evitar scripts maliciosos.

Use filter_var() com filtros apropriados (FILTER_SANITIZE_STRING, FILTER_VALIDATE_EMAIL, etc).

Nunca confie apenas na validação do lado do cliente (JavaScript).

Sempre valide também com PHP.

Em sistemas com login e banco de dados, use prepared statements (PDO) para evitar SQL Injection.

Exemplo completo (GET + POST + validação + sanitização)

```
// formulario.php
<form action="validar.php" method="post">
  <label>Nome:</label>
  <input type="text" name="nome">

  <label>E-mail:</label>
  <input type="email" name="email">

  <button type="submit">Enviar</button>
</form>
```



```
// validar.php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $nome = trim($_POST["nome"]);
    $email = filter_var($_POST["email"],
        FILTER_SANITIZE_EMAIL);

    if (empty($nome) || empty($email)) {
        echo "Preencha todos os campos!";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "E-mail inválido!";
    } else {
        echo "Olá, " . htmlspecialchars($nome)
            . "! Seu e-mail ($email) foi enviado";
    }
}
?>
```

Obrigado!

Quero expressar minha sincera gratidão por ter adquirido este material. Foram meses de trabalho e dedicação para sua finalização. Tenho certeza que este ebook foi de grande importância em sua vida profissional, pois ele foi criado com muito amor, lofi e café.

Lembrando: Este guia de backend ainda não foi finalizado!